



JAVA

By Arijit Chowdhury



THE PRIMITIVE TYPES

Java defines eight primitive types of data: byte, short, int, long, char, float, double, and boolean. The primitive types are also commonly referred to as simple types.

- **Integers** This group includes byte, short, int, and long, which are for whole-valued signed numbers.
- **Floating-point numbers** This group includes float and double, which represent numbers with fractional precision.
- **Characters** This group includes char, which represents symbols in a character set, like letters and numbers.
- **Boolean** This group includes boolean, which is a special type for representing true/false values.

INTEGERS

Java defines four integer types: byte, short, int, and long. All of these are signed, positive and negative values. Java does not support unsigned, positive-only integers. Many other computer languages support both signed and unsigned integers. However, Java's designers felt that unsigned integers were unnecessary. Specifically, they felt that the concept of unsigned was used mostly to specify the behavior of the high-order bit, which defines the sign of an integer value. Java manages the meaning of the high order bit differently, by adding a special "unsigned right shift" operator. Thus, the need for an unsigned integer type was eliminated. The width of an integer type should not be thought of as the amount of storage it consumes, but rather as the behavior it defines for variables and expressions of that type. The Java run-time environment is free to use whatever size it wants, as long as the types behave as you declared them. The width and ranges of these integer types vary widely, as shown in this table:

Name	Width	Range
long	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	-2,147,483,648 to 2,147,483,647
short	16	-32,768 to 32,767
byte	8	-128 to 127

BYTE

The smallest integer type is byte. This is a signed 8-bit type that has a range from –128 to 127. Variables of type byte are especially useful when you're working with a stream of data from a network or file. They are also useful when you're working with raw binary data that may not be directly compatible with Java's other built-in types. Byte variables are declared by use of the byte keyword. For example, the following declares two byte variables called b and c:

```
byte b, c;
```

```
byte[] pixels = new byte[1_000_000]; // Uses 1 MB memory
```

```
int[] pixels = new int[1_000_000]; // Uses 4 MB memory
```

SHORT

short is a signed 16-bit type. It has a range from –32,768 to 32,767. It is probably the least used Java type. Here are some examples of short variable declarations:

```
short s;
```

```
short t;
```

INT

The most commonly used integer type is `int`. It is a signed 32-bit type that has a range from $-2,147,483,648$ to $2,147,483,647$. In addition to other uses, variables of type `int` are commonly employed to control loops and to index arrays. Although you might think that using a `byte` or `short` would be more efficient than using an `int` in situations in which the larger range of an `int` is not needed, this may not be the case. The reason is that when `byte` and `short` values are used in an expression, they are promoted to `int` when the expression is evaluated. Therefore, `int` is often the best choice when an integer is needed.

LONG

long is a signed 64-bit type and is useful for those occasions where an int type is not large enough to hold the desired value. The range of a long is quite large. This makes it useful when big, whole numbers are needed. For example, here is a program that computes the number of miles that light will travel in a specified number of days:

```
// Compute distance light travels using long variables.
class Light {
public static void main(String args[]) {
int lightspeed;
long days;
long seconds;
long distance;
// approximate speed of light in miles per second
lightspeed = 186000;
days = 1000; // specify number of days here
seconds = days * 24 * 60 * 60; // convert to seconds
distance = lightspeed * seconds; // compute distance
System.out.print("In " + days);
System.out.print(" days light will travel about ");
System.out.println(distance + " miles.");
}
}
```

FLOATING-POINT

Floating-point numbers, also known as real numbers, are used when evaluating expressions that require fractional precision. For example, calculations such as square root, or such as sine and cosine, result in a value whose precision requires a floatingpoint type. Java implements the standard (IEEE-754) set of floating-point types and operators. There are two kinds of floating-point types, float and double, which represent single- and double-precision numbers, respectively. Their width and ranges are shown here:

Name	Width in Bits	Approximate Range
double	64	4.9e-324 to 1.8e+308
float	32	1.4e-045 to 3.4e+038

FLOAT

The type float specifies a single-precision value that uses 32 bits of storage. Single precision is faster on some processors and takes half as much space as double precision, but will become imprecise when the values are either very large or very small. Variables of type float are useful when you need a fractional component, but don't require a large degree of precision. For example, float can be useful when representing dollars and cents. Here are some example float variable declarations:

```
float hightemp, lowtemp;
```

DOUBLE

Double precision, as denoted by the double keyword, uses 64 bits to store a value. Double precision is actually faster than single precision on some modern processors that have been optimized for high-speed mathematical calculations. All transcendental math functions, such as `sin( )`, `cos( )`, and `sqrt( )`, return double values. When you need to maintain accuracy over many iterative calculations, or are manipulating large-valued numbers, double is the best choice.

a short program that uses double variables to compute the area of a circle

CHARACTERS

In Java, the data type used to store characters is char. However, C/C++ programmers beware: char in Java is not the same as char in C or C++. In C/C++, char is 8 bits wide. This is not the case in Java. Instead, Java uses Unicode to represent characters. Unicode defines a fully international character set that can represent all of the characters found in all human languages. It is a unification of dozens of character sets, such as Latin, Greek, Arabic, Hebrew, Katakana, Hangul, and many more. At the time of Java's creation, Unicode required 16 bits. Thus, in Java char is a 16-bit type.

The standard set of characters known as ASCII still ranges from 0 to 127 as always, and the extended 8-bit character set

```
class CharDemo {  
    public static void main(String args[]) {  
        char ch1, ch2;  
        ch1 = 88; // code for X  
        ch2 = 'Y';  
        System.out.print("ch1 and ch2: ");  
        System.out.println(ch1 + " " + ch2);  
    }  
}
```

BOOLEANS

Java has a primitive type, called boolean, for logical values. It can have only one of two possible values, true or false. This is the type returned by all relational operators, as in the case of $a < b$. boolean is also the type required by the conditional expressions that govern the control statements such as if and for.

INTEGER LITERALS

Integers are probably the most commonly used type in the typical program. Any whole number value is an integer literal. Examples are 1, 2, 3, and 42. These are all decimal values, meaning they are describing a base 10 number. Two other bases that can be used in integer literals are octal (base eight) and hexadecimal (base 16). Octal values are denoted in Java by a leading zero. Normal decimal numbers cannot have a leading zero. Thus, the seemingly valid value 09 will produce an error from the compiler, since 9 is outside of octal's 0 to 7 range. A more common base for numbers used by programmers is hexadecimal, which matches cleanly with modulo 8 word sizes, such as 8, 16, 32, and 64 bits. You signify a hexadecimal constant with a leading zero-x, (0x or 0X). The range of a hexadecimal digit is 0 to 15, so A through F (or a through f) are substituted for 10 through 15. Integer literals create an int value, which in Java is a 32-bit integer value. Since Java is strongly typed, you might be wondering how it is possible to assign an integer literal to one of Java's other integer types, such as byte or long, without causing a type mismatch error. Fortunately, such situations are easily handled. When a literal value is assigned to a byte or short variable, no error is generated if the literal value is within the range of the target type. An integer literal can always be assigned to a long variable. However, to specify a long literal, you will need to explicitly tell the compiler that the literal value is of type long. You do this by appending an upper- or lowercase L to the literal. For example, 0x7fffffffffffffffL

```
int x = 0b1010;
```

```
int x = 123_456_789;
```

```
int x = 123__456__789;
```

```
int x = 0b1101_0101_0001_1010;
```

FLOATING-POINT LITERALS

Floating-point literals in Java default to double precision. To specify a float literal, you must append an F or f to the constant. You can also explicitly specify a double literal by appending a D or d. Doing so is, of course, redundant. The default double type consumes 64 bits of storage, while the smaller float type requires only 32 bits.

Hexadecimal floating-point literals are also supported, but they are rarely used. They must be in a form similar to scientific notation, but a P or p, rather than an E or e, is used.

For example, 0x12.2P2 is a valid floating-point literal. The value following the P, called the binary exponent, indicates the power-of-two by which the number is multiplied. Therefore, 0x12.2P2 represents 72.5.

```
double num = 9_423_497_862.0;
```

```
double num = 9_423_497.1_0_9;
```

```
float a = 3.14f;
```

```
double b = 2.71;
```

```
double c = 1.2e3;
```

BOOLEAN LITERALS

Boolean literals are simple. There are only two logical values that a boolean value can have, true and false. The values of true and false do not convert into any numerical representation. The true literal in Java does not equal 1, nor does the false literal equal 0. In Java, the Boolean literals can only be assigned to variables declared as boolean or used in expressions with Boolean operators.

STRING LITERALS

String literals in Java are specified like they are in most other languages—by enclosing a sequence of characters between a pair of double quotes. Examples of string literals are

Escape Sequence	Description
<code>\ddd</code>	Octal character (ddd)
<code>\uxxxx</code>	Hexadecimal Unicode character (xxxx)
<code>'</code>	Single quote
<code>"</code>	Double quote
<code>\\</code>	Backslash
<code>\r</code>	Carriage return
<code>\n</code>	New line (also known as line feed)
<code>\f</code>	Form feed
<code>\t</code>	Tab
<code>\b</code>	Backspace

CASTING INCOMPATIBLE TYPES

Although the automatic type conversions are helpful, they will not fulfill all needs. For example, what if you want to assign an int value to a byte variable? This conversion will not be performed automatically, because a byte is smaller than an int. This kind of conversion is sometimes called a narrowing conversion, since you are explicitly making the value narrower so that it will fit into the target type.

To create a conversion between two incompatible types, you must use a cast. A cast is simply an explicit type conversion. It has this general form:
(target-type) value

ARRAYS

One-Dimensional Arrays

type specifies the type of data being allocated, size specifies the number of elements in the array, and array-var is the array variable that is linked to the array. That is, to use new to allocate an array, you must specify the type and number of elements to allocate. The elements in the array allocated by new will automatically be initialized to zero (for numeric types), false (for boolean), or null (for reference types). This example allocates a 12-element array of integers and links them to month_days:

```
int[] month_days = new int[30]; // creates array of size 30, default values = 0
```

```
int[] arr = {10, 20, 30, 40, 50};
```

FOR-EACH LOOP

```
for (type variable : arrayOrCollection) {  
    // body  
}
```

```
int[] arr = {10, 20, 30};
```

```
for (int num : arr) {  
    System.out.println(num);  
}
```

ARRAYS

Multidimensional Arrays

In Java, multidimensional arrays are actually arrays of arrays. These, as you might expect, look and act like regular multidimensional arrays. However, as you will see, there are a couple of subtle differences. To declare a multidimensional array variable, specify each additional index using another set of square brackets. For example, the following declares a twodimensional array variable called twoD:

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
int[][] twoD = new int[4][5];
```

FOR-EACH LOOP

```
for (int[] row : matrix) {  
    for (int element : row) {  
        System.out.print(element + " ");  
    }  
    System.out.println();  
}
```



THANK YOU